Redux Fundamentals

PREREQUISITES

- Familiarity with HTML & CSS.
- Knowledge of React terminology:
  - JSX, Function Components, Props, State, and Hooks

What is Redux?

- Redux is a Javascript library for managing and updating global application state
- Why Should I Use Redux?
  - Redux helps you manage "global" state - state that is needed across many parts of your application.

Integrating Redux with a UI

- Redux is a standalone JS library
- You can use Redux with any UI framework (React, Vue, Angular, Ember, jQuery,…) and use it on both client and server
- Redux was specifically designed to work well with **React**.

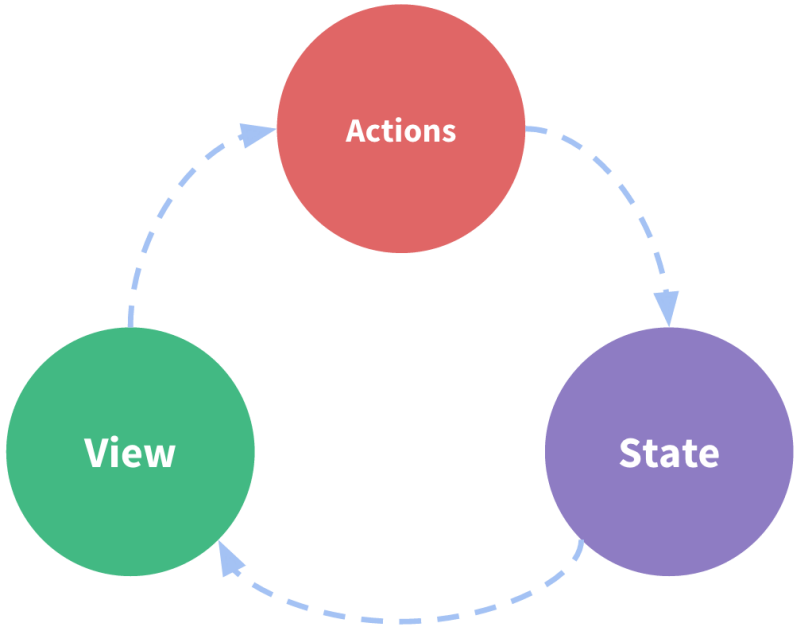
Redux Libraries and Tools

- **Redux Toolkit**
  - recommended approach for writing Redux logic
  - npm install @reduxjs/toolkit
- **React-Redux**
  - npm install react-redux
- **Redux DevTools Extension**

State Management

- *State* describes the condition of the app at a specific point in time

```jsx
import { useState } from 'react';

export function Counter() {
    // State: a counter value
    const [counter, setCounter] = useState(0)

    // Action: code that causes an update to the state when so
    const increment = () => {
      setCounter(prevCounter => prevCounter + 1)
    }

    // View: the UI definition
    return (
      <div>
        Value: {counter} <button onClick={increment}>Increment</button>
      </div>
    )
  }
```



State Management

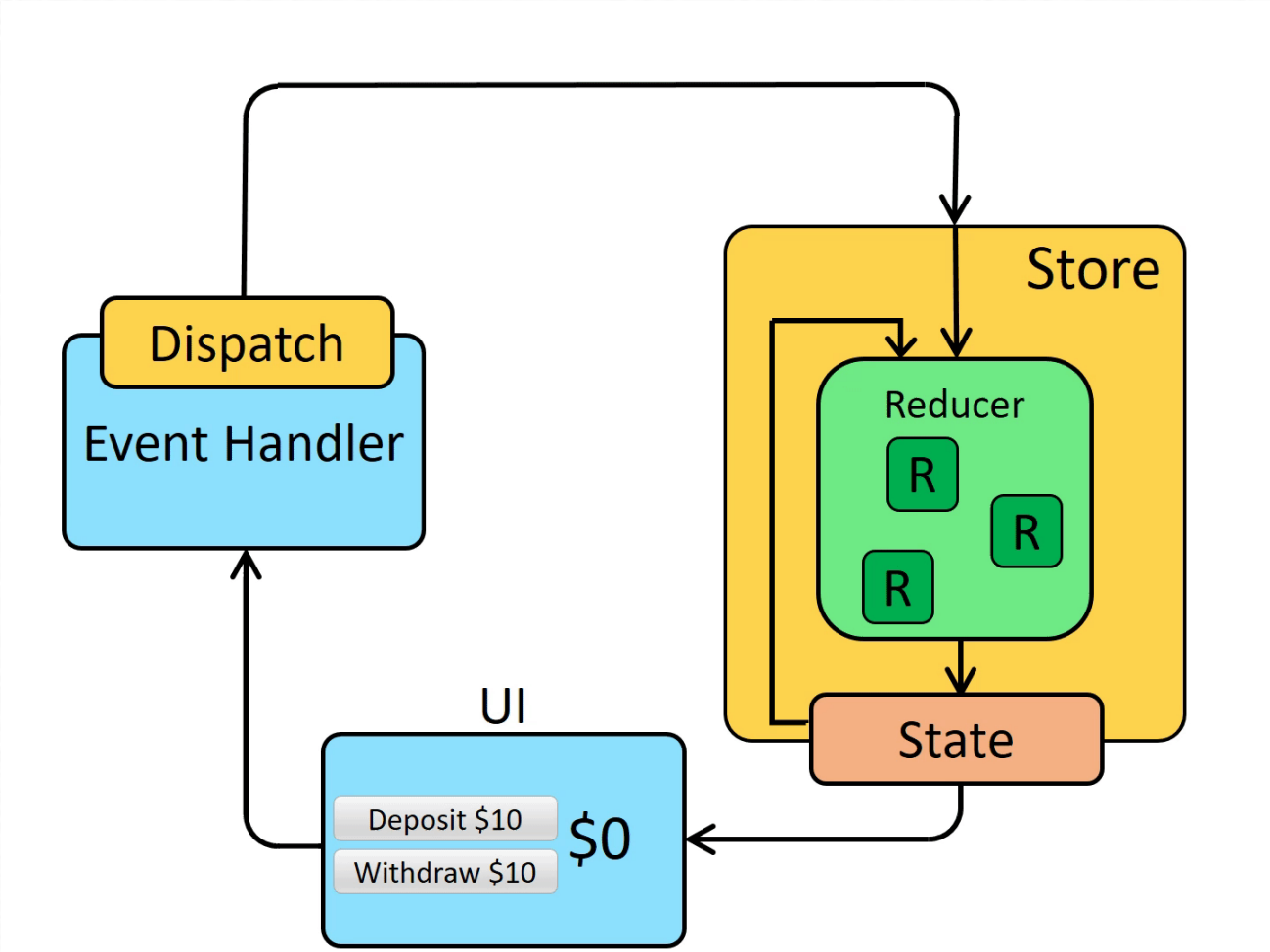
- Multiple components that need to share and use the same state
- The components are located in different parts of the application
- This can be solved by “Lifting state up" to parent components

Immutability

- "Mutable" means "changeable".
- “Immutable", it can never be changed.
- JavaScript objects and arrays are all mutable *by default*
- *React* and *Redux* expect that all state updates are done immutably
- In order to update values immutably,
  - your code must make *copies* of existing objects/arrays, and then modify the copies.

Redux terms

- Actions
- Action Creators
- Reducers
- Store
- Dispatch
- Selectors



Action Object

- An action (event) = a javascript OBJECT
  - type: a string
  - payload: additional information (object)

```
const addTodoAction = {
  type: 'todos/todoAdded',
  payload: 'Buy milk'
}
```

- Action Creators
  - a function that creates and returns an action object

```
const addTodo = text => {
  return {
    type: 'todos/todoAdded',
    payload: text
  }
}
```

Reducers

- A **reducer** is a FUNCTION = an **event listener** = an **event handler**

  receives the current `state` and an `action` object,

  returns the new state: `(state, action) => newState`.

- Example of a reducer

```
const initialState = { value: 0 }

function counterReducer(state = initialState, action) {
  // Check to see if the reducer cares about this action
  if (action.type === 'counter/increment') {
    // If so, make a copy of `state`
    return {
      ...state,
      // and update the copy with the new value
      value: state.value + 1
    }
  }
  // otherwise return the existing state unchanged
  return state
}
```

Store and Dispatch

- A store = a javascript OBJECT = *current* Redux application state
- current state value = storeObject.*getState()*

```
import { configureStore } from '@reduxjs/toolkit'

const store = configureStore({ reducer: counterReducer })

console.log(store.getState())
// {value: 0}
```

- Dispatch(*action object*) = update the state

```
store.dispatch({ type: 'counter/increment' })

console.log(store.getState())
// {value: 1}
```

Action creators and Dispatch

- *Recommended* use Action Creators

```
const increment = () => {
  return {
    type: 'counter/increment'
  }
}

store.dispatch(increment())
```

=

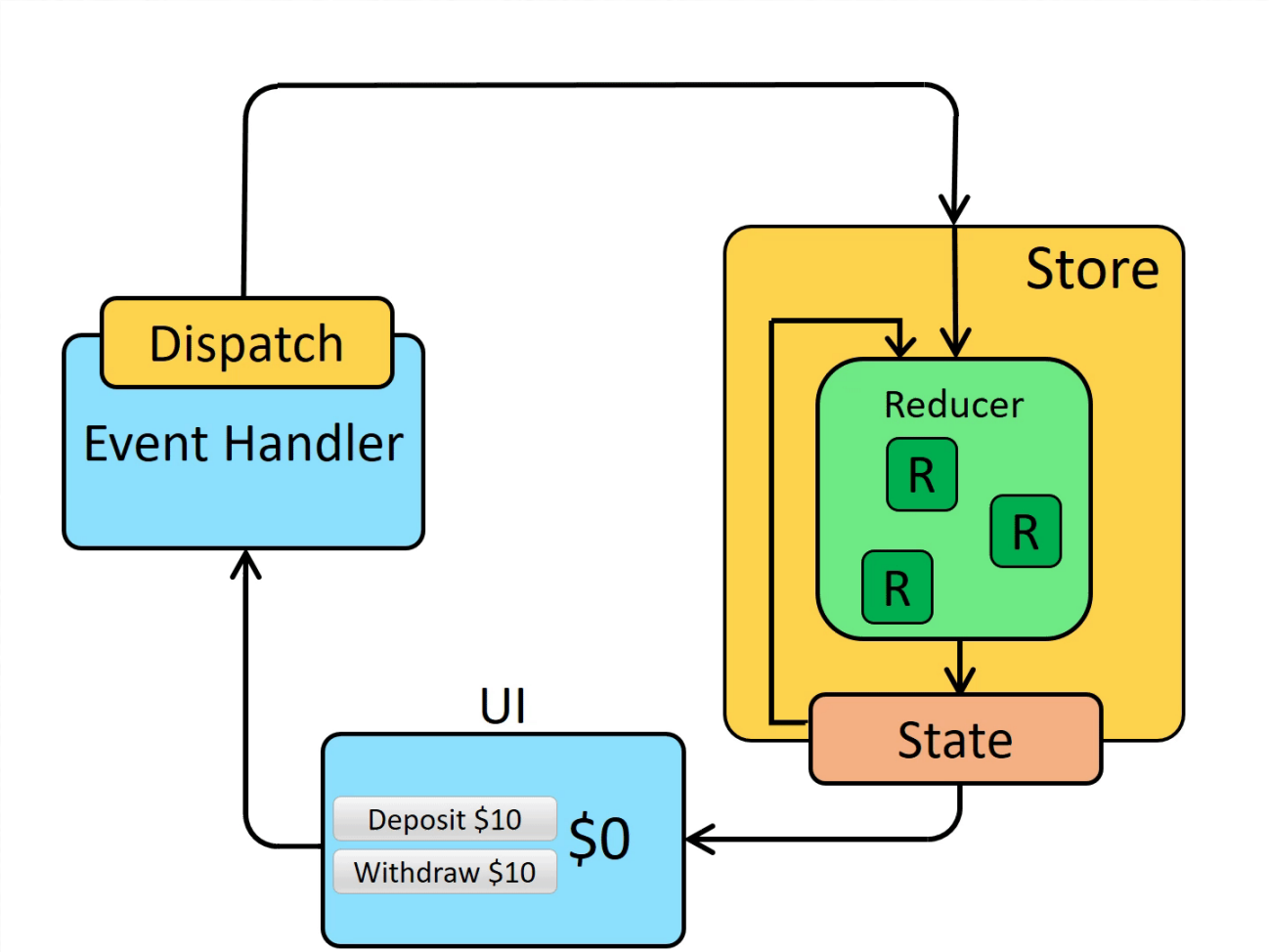
```
store.dispatch({ type: 'counter/increment' })
```

Selectors

- **Selectors** are functions to extract specific pieces of information from a store state value.

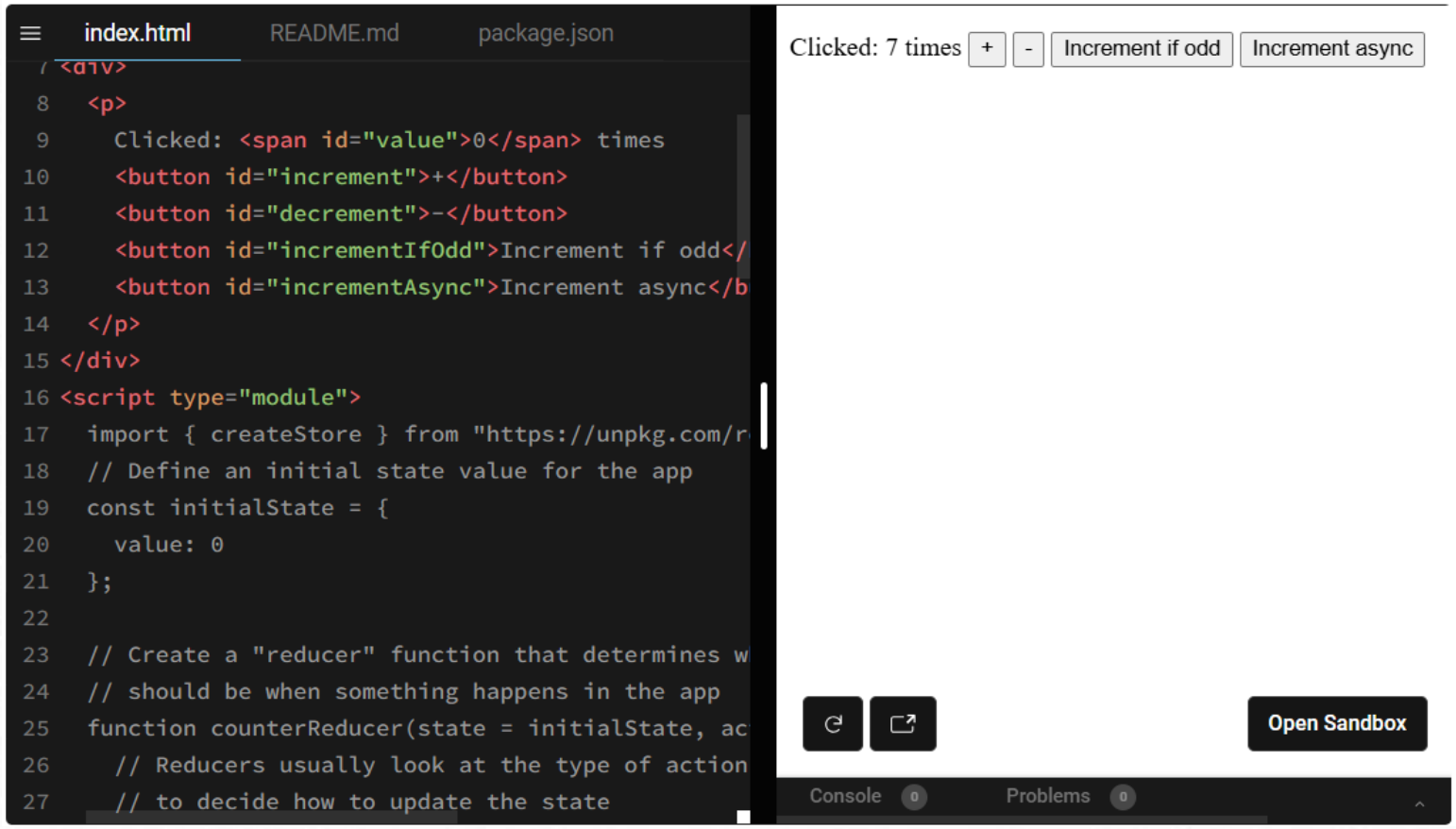
```
const selectCounterValue = state => state.value

const currentValue = selectCounterValue(store.getState())
console.log(currentValue)
// 2
```

Redux Application Data Flow



Redux Core Example App

- The example uses basic *JS and HTML* for the UI
- https://redux.js.org/tutorials/fundamentals/part-1-overview



Redux Toolkit Quick Start

- https://redux.js.org/tutorials/quick-start
- Install Redux Toolkit and React-Redux

```
npm install @reduxjs/toolkit react-redux
```

- Full Counter App Example
  - Open Sandbox

Redux Examples

- https://redux.js.org/introduction/examples